Minishell

GNU Bash manual: HTML (one page) | HTML | PDF

The Open Group: Shell Command Language

Gustavo Junipero y Justin Ennen: Writing Your Own Shell

Otras minishells: J. Castro y M. Parra | A. Pérez y M. Bueno | D. Jiménez e I. Sainz

Guías 42: 42 Cursus Guide

Tu shell deberá:

- Mostrar una entrada mientras espera un comando nuevo.
- Tener un historial funcional.
- Buscar y ejecutar el ejecutable correcto (basado en la variable `PATH` o mediante el uso de rutas relativas o absolutas).
- Evita utilizar más de una variable global para indicar la recepción de una señal. Piensa en lo que implica: Esta aproximación evita que tu gestor de señales acceda a tus estructuras de datos principales.

¡Cuidado! Esta variable global no puede proporcionar ninguna otra información o datos que el número de una señal recibida. Por lo tanto está prohibido utilizar estructuras de tipo "norm" en global.

- No interpretar comillas sin cerrar o caracteres especiales no especificados en el enunciado como `\` (barra invertida) o `;` (punto y coma).
- Gestionar que la `'` evite que el shell interprete los metacaracteres en la secuencia entrecomillada.
- Gestionar que la `"` evite que el shell interprete los metacaracteres en la secuencia entrecomillada exceptuando `$` (signo de dólar).
- Implementar redirecciones:
  - `<` debe redirigir input.
  - `>` debe redirigir output.
  - `>>` debe redirigir el output en modo append.
  - `<<` debe recibir un delimitador, después leer del input de la fuente actual hasta que una línea que contenga solo el delimitador aparezca. Sin embargo, no necesita actualizar el historial.
- Implementar pipes (carácter `|`). El output de cada comando en la pipeline se conecta a través de un pipe al input del siguiente comando.
- Gestionar las variables de entorno (`$` seguidos de caracteres) que deberán expandirse a sus valores.
- Gestionar `$?`, que deberá expandirse al estado de salida del comando más reciente ejecutado en la pipeline.
- Gestionar `Ctrl` + `C`, `Ctrl` + `D` y `Ctrl` + `\`, que deberán funcionar como en bash.
- Cuando sea interactivo:
  - `Ctrl` + `C` imprime una nueva entrada en una línea nueva.
  - `Ctrl` + `D` termina el shell.
  - `Ctrl` + `\` no hace nada.
- Deberá implementar los built-ins:
  - `echo` con la opción `-n`.
  - `cd` solo con una ruta relativa o absoluta.

- `pwd` sin opciones.
- `export` sin opciones.
- `unset` sin opciones.
- `env` sin opciones o argumentos.
- `exit` sin opciones.

Limítate a hacer lo que pide el enunciado. Cualquier cosa no solicitada no se requiere.
Para cada punto, y en caso de dudas, puedes utilizar bash como una referencia.

Funciones autorizadas

Se permite usar `libft`.

stdio.h

```
char    *readline(char *promt);
int      printf(char *format, ...);
```

`readline` reads a line from the terminal and return it using `prompt` as promt.

rl_clear_history

rl_on_new_line

rl_replace_line

rl_redisplay

add_history

https://www.man7.org/linux/man-pages/man3/history.3.html

```
void     add_history(char *string)
```

Place `string` at the end of the history list

stdlib.h

```
void    *malloc(size_t size);
void    *free(void *p);
```

unistd.h

```
ssize_t  read(int fd, void *buf, size_t count);
ssize_t  write(int fd, void *buf, size_t count);
int      access(char *pathname, int mode);
pid_t    fork(void);
void     exit(int status);
int      unlink(char *path);
int      execve(char *path, char *argv[], char *envp[]);
int      dup(int oldfd);
int      dup2(int oldfd, int newfd);
int      pipe(int pipefd[2]);
char    *getcwd(char buf[size], size_t size);
int      chdir(char *path);
```

fcntl.h

```
int      open(char *pathname, int flags);
int      close(int fd);
```

sys/wait.h

```
pid_t    wait(int *wstatus);
```

```
pid_t    waitpid(pid_t pid, int *wstatus, int options);
```

wait3
wait4

signal.h

signal: signal(2), signal(7) (creo que es signal(2))
sigaction(2)
int kill(pid_t pid, int sig)
Send signal to a process

sys/stat.h

```
int      stat(char *path, struct stat statbuf);
int      lstat(char *path, struct stat statbuf);
int      fstat(int fd, struct stat statbuf);
```

These functions return information about a file, in the buffer pointed to by statbuf

getcwd
copies an absolute pathname of the current working directory to the array pointed to by buf, which is of length size.

chdir
Changes the current working directory of the calling process to the directory specified in path

opendir
The opendir() function opens a directory stream corresponding to the directory name, and returns a pointer to the directory stream. The stream is positioned at the first entry in the directory.
readdir
The readdir() function returns a pointer to a dirent structure representing the next directory entry in the directory stream pointed to by dirp.
closedir
closes the directory stream associated with dirp. A successful call to closedir() also closes the underlying file descriptor associated with dirp

ERRORES
strerror
perror

TERMINAL
isatty
Tests whether fd is an open file descriptor referring to a terminal. Returns 1 if fd is an open file descriptor referring to a terminal; otherwise 0 is returned, and errno is set to indicate the error.
ttyname

returns a pointer to the null-terminated pathname of the terminal device that is open on the file descriptor fd, or NULL on error (for example, if fd is not connected to a terminal).
ttyslot
returns the index of the current user's entry in some file.
ioctl
manipulates the underlying device parameters of special files. In particular, many operating characteristics of character special files (e.g., terminals) may be controlled with ioctl() operations

getenv
searches the environment list to find the environment variable name, and returns a pointer to the corresponding value string

tcsetattr
tcgetattr
tgetent
tgetflag
tgetnum
tgetstr
tgoto
tputs

Funciones

readline

https://man7.org/linux/man-pages/man3/readline.3.html

```
char    *readline(char *promt);
```

`readline` reads a line from the terminal and return it, using `prompt` as a prompt. If `prompt` is `NULL` or the empty string, `readline` does not issue a prompt.
The line returned is allocated with `malloc(3)`; the caller must free it when finished.
The line returned has the final newline removed, so only the text of the line remains.

Para compilar hay que añadir `-lreadline`:

```
#include <stdio.h>
#include <stdlib.h>
#include <readline/readline.h>
#include <readline/history.h>

int    main(void)
{
    char    *rl;

    rl = readline("$ ");
    printf("%s\n", rl);
    free(rl);
    return (0);
}
```

rl_clear_history

```
void rl_clear_history(void);
```

The `rl_clear_line()` function clears the history list by deleting all of the entries. The `rl_clear_line()` function frees data that the `readline` library saves in the histroy list.

rl_on_new_line

```
int rl_on_new_line(void);
```

The rl_on_new_line() function tells the update routine that we have moved onto a new empty line, usually used after outputting a line.

rl_replace_line

I didn't find any information on that function.

rl_redisplay

```
int rl_redisplay(void);
```

The `rl_redisplay()` change what's displayed on the screen to reflect the current contents of `rl_line_buffer`.

add_history

```
void add_history(char *s);
```

The `add_history()` function saves the line passed as parameter in the history so it can be retrieved later in the terminal (like pressing the up arrow in bash).

getcwd

https://man7.org/linux/man-pages/man3/getcwd.3.html

```
char *getcwd(char *buf, size_t size);
```

The `getcwd()` returns a null-terminated string containing the absolute pathname that is the current working directory of the calling process. The pathname is returned as the function result and via the argument `buf`.

Ejemplo de programa

```
int    main(void)
{
    char    *directory;

    directory = getcwd(NULL, 0);
    printf("%s\n", directory);
    free(directory);
    return (0);
}
```

chdir

https://man7.org/linux/man-pages/man2/chdir.2.html

```
int chdir(const char *path);
```

`chdir()` changes the current working directory of the calling process to the directory specified in `path`.

```
int    main(void)
{
    char    *pwd;

    pwd = getcwd(NULL, 0);
    printf("%s\n", pwd);        // /home/danielji/Documents/minishell
    free(pwd);
    chdir("/home/danielji/Documents/philosophers");
    pwd = getcwd(NULL, 0);
    printf("%s\n", pwd);        // /home/danielji/Documents/philosophers
    free(pwd);
    return (0);
}
```

stat, lstat, fstat

```
#include <sys/stat.h>
int stat(char *pathname, struct stat *statbuf);
int lstat(char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
```

These functions return information about a file in the structure pointed to by *statbuf*.

opendir

https://man7.org/linux/man-pages/man3/opendir.3.html

```
#include <sys/types.h>
#include <dirent.h>
DIR *opendir(char *name);
```

The `opendir()` function opens a directory stream corresponding to the directory `name`, and returns a pointer to the directory stream. The stream is positioned at the first entry in the directory.

readdir

https://man7.org/linux/man-pages/man3/readdir.3.html

```
#include <dirent.h>
struct dirent *readdir(DIR *dirp);
```

The `readdir()` function returns a pointer to a dirent structure representing the next directory entry in the directory stream pointed to by `dirp`. It returns `NULL` on reaching the end of the directory stream or if an error occured.

closedir

https://man7.org/linux/man-pages/man3/closedir.3.html

```
#include <sys/types.h>
#include <dirent.h>
int closedir(DIR *dirp);
```

The `closedir()` function closes the directory stream associated with `dirp`. A successful call to `closedir()` also closes the underlying file descriptor associated with `dirp`.
The directory stream descriptor `dirp` is not available after this call.

strerror

https://man7.org/linux/man-pages/man3/strerror.3.html

```
#include <string.h>
char *strerror(int errnum);
```

The `strerror()` function returns a pointer to a string that describes the error code passed in the argument `errnum`. This string must not be modified by the application, but may be modified by a subsequent call to `strerror()` or `strerror_l()`. No other library function, including `perror()`, will modify this string.

perror

```
#include <stdio.h>
void perror(const char *s);
```

The `perror()` function produces a message on standard error describing the last error encountered during a call to a system or library function.

isatty

https://man7.org/linux/man-pages/man3/isatty.3.html

```
#include <unistd.h>
int isatty(int fd);
```

The `isatty` function tests wether `fd` is a terminal.

ttyname

https://man7.org/linux/man-pages/man3/ttyname.3.html

```
#include <unistd.h>
char **ttyname(int fd);
```

The `ttyname()` function returns a pointer to the null-terminated pathname of the terminal device that is open on the file descriptor `fd`, or `NULL` on error.

ttyslot

https://man7.org/linux/man-pages/man3/ttyslot.3.html

```
#include <unistd.h>
int ttyslot(void);
```

This is a legacy function with some backstory.

ioctl

https://man7.org/linux/man-pages/man2/ioctl.2.html

```
#include <sys/ioctl.h>
int ioctl(int fd, unsigned long request, ...);
```

The `ioctl()` system call manipulates the underlying device parameters of a special files.

getenv

https://man7.org/linux/man-pages/man3/getenv.3.html

```
#include <stdlib.h>
char *getenv(const char *name);
```

The `getenv()` function searches the environment list to find the environment variable `name`, and returns a pointer to the corresponding value string.

tcsetattr

https://man7.org/linux/man-pages/man3/tcsetattr.3p.html

```
#include <termios.h>
int tcsetattr(int fildes, int optional_actions, const struct *termios_p);
```

The `tcsetattr()` function shall set the parameters associated with the terminal referred to by the open file descriptor `fildes` from the termios structure referenced by `termios_p`.

tcgetattr

https://man7.org/linux/man-pages/man3/tcgetattr.3p.html

```
#include <termios.h>
int tcgetattr(int fildes, struct termios *termios_p);
```

The `tcgetattr()` function shall get the parameters associated with with the terminal reffered to by `fildes` and store them in the termios structure referenced by `termios_p`.

tgetent

https://linux.die.net/man/3/tgetent

```
#include <curses.h>
#include <term.h>

int tgetent(char *bp, const char *name);
int tgetflag(char *id);
int tgetnum(char *id);
char *tgetstr(char *id, char **area);
char *tgoto(const char *cap, int col, int row);
int tputs(const char *str, int affcnt, int (*putc)(int));
```

These routines are included as a conversion aid for programs that use the termcap library.

Cómo hacer la minishell

Una minishell es un procesador de textos que ejecuta comandos. Es también un lenguaje de programación, pero esa parte no entra dentro de los objetivos de la minishell.
Una shell también contiene programas integrados (*builtins*), de los cuales debemos implementar algunos: `echo`, `cd`, `pwd`, `export`, `unset`, `env`, `exit`.

Dos partes:
- Lexer / parser / expander
- Executer

Cada una de estas partes tiene, a su vez, otras partes.

Algunas definiciones

Según Bash Reference Manual:

- **token**: Secuencia de caracteres que la shell considera una sola unidad. Puede ser un **word** o un **operator**.

- **word**: Secuencia de caracteres que la shell trata como una sola unidad. Una word no incluye metacaracteres sin comillas. (Si una word tiene metacaracteres, tienen que estar entrecomillados).

- **operator**: Puede ser un **control operator** o un **redirection operator**. Contienen al menos un metacarácter sin comillas.
  - **control operator**: Un token como `\n`, `||`, `&&`, `&`, `;`, `;;`, `;&`, `;;&`, `|`, `|&`, `(`, `)`.
  - **redirection operator**: Un token como `<`, `>`, `>>`, etc.

- **metacharacter**: Un carácter que, cuando no está entrecomillado, separa palabras. Puede ser un espacio, tab, newline o `|`, `&`, `;`, `(`, `)`, `<`, `>`.

- **name**: Un name o identifier es una word compuesta por letras, números o `_`. Debe comenzar por una letra o `_`. Se usan para nombrar variables y funciones.

Según Wikipedia:

- **lexer**: Programa o función que convirte texto en tokens con significado léxico.
  - identifiers
  - operators: `+`, `<`, `=`
  - grouping symbols: `{}`, `()`, `;`
  - data types: `int`, `char`
  - language keywords: `if`, `while`, `return`, `for`

- **parser**: Programa o función que construye una estructura de datos a partir de una entrada de texto. Estas estructuras suelen ser jerárquicas (árboles de sintaxis, AST, tablas de comandos, etc.).

Operación de una shell

Según Bash Reference Manual.

The shell reads its input and divides the input into **words** and **operators**, employing the **quoting rules** to select which meanings to assign various words and characters.

The shell then parses these tokens into commands and other constructs, removes the special meaning of certain words or characters, expands others, redirects input and output as needed, executes the specified command, waits for the command's exit status, and makes that exit status available for further inspection or processing

Basically, the shell does the following:

- Reads its input from [...] the user's terminal.
- Breaks the input into words and operators, obeying the quoting rules described in Quoting. These tokens are separated by metacharacters. This step performs alias expansion (see Aliases).
- Parses the tokens into simple and compound commands (see Shell Commands).
- Performs the various shell expansions (see Shell Expansions), breaking the expanded tokens into lists of filenames (see Filename Expansion) and commands and arguments.
- Performs any necessary redirections (see Redirections) and removes the redirection operators and their operands from the argument list.
- Executes the command (see Executing Commands).
- Optionally waits for the command to complete and collects its exit status (see Exit Status).

Implementación de A. Pérez y M. Bueno

https://github.com/madebypixel02/minishell

Primer paso: Lexer y Expander

Se separa el string del input por espacios y comillas. Por ejemplo:

```
echo "hello      there" how are 'you 'doing? $USER |wc -l >outfile

    ↓

echo "hello      there" how are 'you 'doing? $USER |wc -l >outfile

    ↓

{ echo, "hello      there", how, are, 'you 'doing?, $USER, |wc, -l, >outfile, NULL }
```

A cada substring se le aplican las funciones del expander para transformar las variables (`$` y `~`).
Si no se encuentra una variable se reemplaza por un string vacío.

```
{ echo, "hello      there", how, are, 'you 'doing?, $USER, |wc, -l, >outfile, NULL }

    ↓

{ echo, "hello      there", how, are, 'you 'doing?, danielji, |wc, -l, >outfile, NULL }
```

Por último se hace una separación por `<` , `|` y `>` pero sólo si esos caracteres están fuera de comillas:

```
{ echo, "hello      there", how, are, 'you 'doing?, danielji, |wc, -l, >outfile, NULL }

    ↓

{ echo, "hello      there", how, are, 'you 'doing?, danielji, |, wc, -l, >, outfile, NULL }
```

Segundo paso: Parser

En este paso se almacenan los strings tokenizados de forma que sean útiles para la ejecución.
Por ejemplo:

```
cmds:
    cmd 1:
        infile:    0                    # default
        outfile:   1                    # redirected to pipe
        full_path: NULL                 # because echo is a builtin
        full_cmd:  {echo, hello there, how, are, you doing?, danielji, NULL}
    cmd 2:
        infile:    0                    # contains output of previous command
        outfile:   3                    # fd corresponding to the open file 'outfile'
        full_path: /bin/wc
        full_cmd:  {wc, -l, NULL}
envp:                                   # (envp from main)
pid:                                    # process ID of current instance
g_status: 0                             # (if last command exits normally)
```

Tercer paso: Executer

Se ejecutan los comandos como en Pipex o los builtins.
Se busca la ruta del ejecutable si es necesario.

Otro ejemplo:

```
<Makefile cat| echo "$PWD 'hola'" ~/src | 'tr' -d / >outfile

    ↓

<Makefile cat| echo "$PWD 'hola'" ~/src | 'tr' -d / >outfile

    ↓

{ <Makefile, cat|, echo, "$PWD 'hola'", ~/src, |, 'tr', -d, /, >outfile, NULL }

    ↓

{ <Makefile, cat|, echo, "/home/danielji/minishell 'hola'", /home/danielji/src, |, 'tr', -d,
/, >outfile, NULL }

    ↓

{ <, Makefile, cat, |, echo, "/home/danielji/minishell 'hola'", /home/danielji/src, |, 'tr',
-d, /, >, outfile, NULL }

    ↓

{ <, Makefile, cat, |, echo, /home/danielji/minishell 'hola', /home/danielji/src, |, tr, -d,
/, >, outfile, NULL }

    ↓

cmd 1:
    full_cmd:  cat
    full_path: cat
    pid:       -1
    infile:    3
    outfile:   1

cmd 2:
    full_cmd:  echo ; /home/danielji/minishell 'hola' ; /home/danielji/src
    full_path: echo
    pid:       -1
    infile:    0
    outfile:   1

cmd 3:
    full_cmd:  tr ; -d ; /
    full_path: tr
    pid:       -1
    infile:    0
    outfile:   4
```